

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

Институт №8 «Компьютерные науки и прикладная математика»

**Лабораторная работа №2
по курсу «Технологии параллельного программирования»**

Работа с матрицами. Метод Гаусса.

Выполнил: О.С. Концебалов

Группа: М8О-409Б-22

Преподаватели: А.Ю. Морозов,
Е.Е. Заяц

Москва, 2025

Условие

Цель работы: использование объединения запросов к глобальной памяти. Реализация метода Гаусса с выбором главного элемента по столбцу. Ознакомление с библиотекой для параллельных расчетов Thrust. Использование *двухмерной сетки потоков*. Исследование производительности программы с помощью утилиты nvprof (*обязательно отразить в отчете*).

В качестве вещественного типа данных необходимо использовать тип данных double. Библиотеку Thrust использовать только для поиска максимального элемента на каждой итерации алгоритма. В вариантах (1, 5, 6, 7), где необходимо сравнение по модулю с нулем, в качестве нулевого значения использовать 10^{-7} . Все результаты выводить с абсолютной точностью 10^{-10} .

Вариант 2. Вычисление обратной матрицы.

Входные данные: на первой строке задано число n – размер матрицы. В следующих n строках, записано n вещественных чисел – элементы матрицы. $n \leq 10^4$.

Выходные данные: необходимо вывести на n строках, по n чисел – элементы обратной матрицы.

Пример:

Входной файл	Выходной файл
2	-2.0000000000e+00 1.0000000000e+00
1 2	1.5000000000e+00 -5.0000000000e-01
3 4	

Программное и аппаратное обеспечение

OS

Семейство	Windows
Версия	10.0.19045
Релиз	10
Платформа	Windows-10-10.0.19045
Архитектура	AMD64

CPU

Модель	AMD Ryzen 5 5600H with Radeon Graphics
Архитектура	AMD64
Разрядность	64
Частота (паспортная)	3.2940 GHz
Частота (факт.)	3.3010 GHz
Ядер (физ.)	6
Потоков (лог.)	12

RAM

Всего	15.35 GB
Доступно	4.35 GB
Swap	6.00 GB

Диски

Диск	ФС	Всего, GB	Использовано, GB	Свободно, GB	Использовано, %
C:\	NTFS	470.94	391.13	79.82	83.05
D:\	FAT32	0.50	0.00	0.50	0.00

GPU

Compute capability	8.6
Name	NVIDIA GeForce RTX 3050 Ti Laptop GPU
Total Global Memory	4294443008
Shared memory per block	49152
Registers per block	65536
Warp size	32
Max threads per block	(1024, 1024, 64)
Max block	(2147483647, 65535, 65535)
Total constant memory	65536
Multiprocessor's count	20

Используемая IDE: Visual Studio Code

Версия компилятора:

```
C:\Program Files\Microsoft Visual Studio\2022\Community>nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Wed_Aug_20_13:58:20_Pacific_Daylight_Time_2025
Cuda compilation tools, release 13.0, V13.0.88
Build cuda_13.0.r13.0/compiler.36424714_0
```

Метод решения

В программе реализован метод Гаусса–Жордана для нахождения обратной матрицы на GPU. Из стандартного ввода считывается размер матрицы n , затем её элементы; исходная матрица A хранится в `std::vector<double>` длины $n \cdot n$ в столбцовом формате (элемент с координатами (row, column) лежит по индексу $(column * n + row)$). Параллельно создаётся вторая матрица E того же размера, инициализируемая как единичная (аналог расширенной матрицы $[A|I]$). Обе матрицы копируются на видеокарту в буферы `gruA` и `gruE` с помощью `cudaMalloc` и `cudaMemcpy`, все CUDA-вызовы обернуты макросом `CSC` с проверкой кода ошибки.

Прямой ход метода (обнуление элементов под диагональю) выполняется в цикле по столбцам $i=0, \dots, n-2$. На каждом шаге с помощью библиотеки Thrust на устройстве выбирается строка с максимальным по модулю элементом в текущем столбце. Если найденная строка отличается от текущей, вызывается ядро `kernelSwapRows`, которое параллельно по всем столбцам меняет местами строки в массивах `gruA` и `gruE`. Затем

запускается ядро `kernelUnderDiagonale`: для фиксированной опорной строки x оно по строкам ниже диагонали и вычисляет множители $k = -A_{xi}/A_{xx}$, добавляя k -кратную опорную строку ко всем строкам ниже. Те же элементарные преобразования одновременно применяются к матрице E .

После завершения прямого хода выполняется обратный ход — обнуление элементов над диагональю. В цикле по $x = n-1, n-2, \dots, 1$ запускается ядро `kernelUpperDiagonale`, которое, используя коэффициенты из текущего столбца матрицы `gruA`, вычитает подходящие линейные комбинации опорной строки из всех строк выше, но уже только для матрицы `gruE`. На заключительном этапе ядро `kernelScaleRhs` пробегает по строкам и делит каждую строку E на соответствующий диагональный элемент A_{ii} , тем самым нормируя диагональ до единиц и получая в `gruE` готовую обратную матрицу A^{-1} . Все вычислительные ядра используют двухмерную сетку блоков `grid(64,64)` и `block(32,8)`, внутри которой отдельные потоки обрабатывают свои подмассивы с шагами по осям sx и sy , что позволяет масштабировать алгоритм на произвольный размер n . По завершении матрица `gruE` копируется обратно на CPU, и результат выводится в стандартный поток в экспоненциальном формате с заданной точностью, после чего GPU-память освобождается.

Описание программы

Программа состоит из функции `main`, четырёх CUDA-ядер (`kernelSwapRows`, `kernelUnderDiagonale`, `kernelUpperDiagonale`, `kernelScaleRhs`), вспомогательного компаратора `Comparator` для Thrust и макроса `CSC` для проверки ошибок. Матрицы A и E хранятся в виде одномерных массивов `double` длины $n \times n$. Размер сетки потоков задаётся глобальными константами `block = (32,8)` и `grid = (64,64)` и используется для всех ядер.

Ядра.

`kernelSwapRows` параллельно меняет местами две строки матриц A и E . Потоки формируют глобальный линейный индекс `tid` по сетке `grid×block` и, шагая с шагом `stride = threadsX×threadsY`, бегут по столбцам; для каждого столбца `column` переставляются элементы `[column*n + i]` и `[column*n + j]` и в A , и в E . `kernelUnderDiagonale` реализует прямой ход: для фиксированного столбца/строки x потоки по оси x перебирают строки $i > x$ с шагом sx , вычисляют множитель $k = -A[x*n + i] / A[x*n + x]$ и обновляют элементы той же строки i в A и E , добавляя k -кратную опорную строку x . `kernelUpperDiagonale` выполняет обратный ход: для фиксированной строки x потоки по оси x проходят по строкам $i < x$ и с тем же коэффициентом k обнуляют элементы выше диагонали, но уже только в матрице E (матрица A используется как «замороженный» треугольник). `kernelScaleRhs` завершает метод: для каждой строки `row` извлекается диагональный элемент $d = A[row*n + row]$, после чего все элементы `E[column*n + row]` делятся на d — диагональ становится единичной, а E превращается в A^{-1} .

```

__global__ void kernelSwapRows(double *A, double *E, int n, int i, int j) {
    size_t threadsX = blockDim.x * gridDim.x;
    size_t threadsY = blockDim.y * gridDim.y;

    size_t gx      = blockIdx.x * blockDim.x + threadIdx.x;
    size_t gy      = blockIdx.y * blockDim.y + threadIdx.y;
    size_t tid     = gy * threadsX + gx;
    size_t stride  = threadsX * threadsY;

    for (size_t column = tid; column < n; column += stride) {
        // A
        double tmp = A[column * n + i];
        A[column * n + i] = A[column * n + j];
        A[column * n + j] = tmp;

        // E
        tmp = E[column * n + i];
        E[column * n + i] = E[column * n + j];
        E[column * n + j] = tmp;
    }
}

__global__ void kernelScaleRhs(const double* A, double* E, int n) {
    size_t ix = blockIdx.x * blockDim.x + threadIdx.x;
    size_t iy = blockIdx.y * blockDim.y + threadIdx.y;
    size_t sx = gridDim.x * blockDim.x;
    size_t sy = gridDim.y * blockDim.y;

    for (size_t row = ix; row < n; row += sx) {
        double d = A[row * n + row];

        for (size_t column = iy; column < n; column += sy) {
            E[column * n + row] /= d;
        }
    }
}

// Прямой ход - обнуление элементов под диагональю
__global__ void kernelUnderDiagonale(double* A, double* E, int n, int x)
{
    size_t ix = blockIdx.x * blockDim.x + threadIdx.x;
    size_t iy = blockIdx.y * blockDim.y + threadIdx.y;
    size_t sx = gridDim.x * blockDim.x;
    size_t sy = gridDim.y * blockDim.y;

    for (size_t i = x + 1 + ix; i < n; i += sx) {
        double cur = -A[x * n + i];
        double d = A[x * n + x];
        double k = cur / d;

        for (size_t j = x + 1 + iy; j < n; j += sy) {
            A[j * n + i] += k * A[j * n + x];
        }

        for (size_t j = iy; j < n; j += sy) {
            E[j * n + i] += k * E[j * n + x];
        }
    }
}

```

```
// Обратный ход - обнуление выше диагонали
__global__ void kernelUpperDiagonale(const double* A, double* E, int n, int x) {
    size_t ix = blockIdx.x * blockDim.x + threadIdx.x;
    size_t iy = blockIdx.y * blockDim.y + threadIdx.y;

    size_t sx = blockDim.x;
    size_t sy = blockDim.y;

    for (int64_t i = x - 1 - ix; i >= 0; i -= sx) {
        double cur = -A[x * n + i];
        double d = A[x * n + x];
        double k = cur / d;

        for (int j = iy; j < n; j += sy) {
            E[j * n + i] += k * E[j * n + x];
        }
    }
}
```

Проверка ошибок.

Для всех CUDA-вызовов используется макрос CSC(...), который проверяет код возврата, а при ошибке выводит файл/строку и текст сообщения драйвера.

```
// Check CUDA call macros
#define CSC(call)
do {
    cudaError_t res = call;
    if (res != cudaSuccess) {
        fprintf(stderr, "ERROR in %s:%d. Message: %s\n",
            __FILE__, __LINE__, cudaGetErrorString(res));
        exit(0);
    }
} while(0)
```

Функция main.

В main сначала считывается размер n , затем элементы матрицы A из `std::cin` построчно, но кладутся в вектор в столбцовом виде ($A[\text{column} * n + \text{row}]$). Параллельно создаётся матрица E размера $n \times n$, инициализируемая как единичная. Выделяются два буфера на GPU (`gpuA`, `gpuE`), в которые копируются A и E . Далее выполняется прямой ход: для $i = 0..n-2$ выбирается строка с максимальным по модулю элементом в столбце i , при необходимости вызывается `kernelSwapRows`, затем стартует `kernelUnderDiagonale<<<grid, block>>>(gpuA, gpuE, n, i)`. После этого запускается обратный ход по $i = n-1..1$ с ядром `kernelUpperDiagonale`, и в финале — `kernelScaleRhs` для нормировки диагонали. Готовая обратная матрица E копируется с устройства на хост, выводится в `std::cout` в экспоненциальном формате (`std::scientific, precision(10)`) построчно и с пробелами между элементами, после чего GPU-память освобождается (`cudaFree(gpuA), cudaFree(gpuE)`), и программа завершает работу.

```

int main() {
    int n;
    std::cin >> n;

    std::vector<double> A(n * n);
    for (size_t row = 0; row < n; ++row) {
        for (size_t column = 0; column < n; ++column) {
            std::cin >> A[column * n + row];
        }
    }

    std::vector<double> E(n * n);
    for (size_t row = 0; row < n; ++row) {
        for (size_t column = 0; column < n; ++column) {
            E[column * n + row] = ((row == column) ? 1.0 : 0.0);
        }
    }

    double *gpuA;
    double *gpuE;

    CSC(cudaMalloc(&gpuA, n * n * sizeof(double)));
    CSC(cudaMalloc(&gpuE, n * n * sizeof(double)));
    CSC(cudaMemcpy(gpuA, A.data(), n * n * sizeof(double), cudaMemcpyHostToDevice));
    CSC(cudaMemcpy(gpuE, E.data(), n * n * sizeof(double), cudaMemcpyHostToDevice));

    const thrust::device_ptr<double> ptr = thrust::device_pointer_cast(gpuA);
    const Comparator comparator;

    // Прямой ход
    for (size_t i = 0; i < n - 1; ++i) {
        auto begin = ptr + i * n + i;
        auto end = ptr + i * n + n;

        auto it = thrust::max_element(thrust::device, begin, end, comparator);
        int maxElemIndex = (it - ptr) - i * n; // индекс строки

        if (maxElemIndex != i) {
            kernelSwapRows<<<grid, block>>>(gpuA, gpuE, n, i, maxElemIndex);
        }

        kernelUnderDiagonale<<<grid, block>>>(gpuA, gpuE, n, i);
    }

    // Обратный ход
    for (int64_t i = n - 1; i > 0; --i) {
        kernelUpperDiagonale<<<grid, block>>>(gpuA, gpuE, n, i);
    }

    kernelScaleRhs<<<grid, block>>>(gpuA, gpuE, n);

    CSC(cudaMemcpy(E.data(), gpuE, n * n * sizeof(double), cudaMemcpyDeviceToHost));
    CSC(cudaFree(gpuA));
    CSC(cudaFree(gpuE));

    std::cout << std::scientific;
    std::cout.precision(10);
}

```

```
for (size_t row = 0; row < n; ++row) {
    for (size_t column = 0; column < n; ++column) {
        if (column) {
            std::cout << ' ';
        }
        std::cout << E[column * n + row];
    }

    std::cout << '\n';
}

return 0;
}
```

Результаты

Для измерения производительности тесты проводились на матрице размером 4000 на 4000

Конфигурация	Время, s
<<<1024, 1024>>>	8.5294
<<<512, 512>>>	8.2705
<<<256, 1>>>	9.9436
<<<64, 32>>>	8.1947
<<<8, 2>>>	9.1434

CPU time = **467.9214 s**

Профилировщик запускался на матрице размером 100 на 100. На больших размерах он падал с ошибкой, хотя сам код отработывает корректно. Разобраться в чем проблема не удалось, поэтому было принято решение использовать матрицу меньшей размерности

kernelSwapRows

kernelSwapRows(double *, double *, int, int, int) (64, 64, 1)x(32, 8, 1), Context 1, Stream 7, Device 0, CC 8.6

Section: Command line profiler metrics

Metric Name	Metric Unit	Metric Value
gpu__time__duration.sum	usecond	31,10

l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.avg		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.max		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.min		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.avg		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.max		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.min		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum		0
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.avg	sector	689,13
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.max	sector	1 248
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.min	sector	425
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum	sector	21 839
l1tex__t_sectors_pipe_lsu_mem_global_op_st.avg	sector	170,12
l1tex__t_sectors_pipe_lsu_mem_global_op_st.max	sector	489
l1tex__t_sectors_pipe_lsu_mem_global_op_st.min	sector	0
l1tex__t_sectors_pipe_lsu_mem_global_op_st.sum	sector	5 256
sm__sass_inst_executed_op_local.avg	inst	0
sm__sass_inst_executed_op_local.max	inst	0
sm__sass_inst_executed_op_local.min	inst	0
sm__sass_inst_executed_op_local.sum	inst	0
smsp__branch_targets_threads_divergent		0

kernelUnderDiagonale

kernelUnderDiagonale(double *, double *, int, int) (64, 64, 1)x(32, 8, 1), Context 1,
Stream 7, Device 0, CC 8.6

Section: Command line profiler metrics

Metric Name	Metric Unit	Metric Value

gpu__time_duration.sum	usecond	38,66
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.avg		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.max		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.min		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.avg		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.max		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.min		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum		0
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.avg	sector	746,60
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.max	sector	1 322
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.min	sector	376
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum	sector	22 398

l1tex__t_sectors_pipe_lsu_mem_global_op_st.avg	sector	185,73
l1tex__t_sectors_pipe_lsu_mem_global_op_st.max	sector	579
l1tex__t_sectors_pipe_lsu_mem_global_op_st.min	sector	0
l1tex__t_sectors_pipe_lsu_mem_global_op_st.sum	sector	5 572
sm__sass_inst_executed_op_local.avg	inst	0
sm__sass_inst_executed_op_local.max	inst	0
sm__sass_inst_executed_op_local.min	inst	0
sm__sass_inst_executed_op_local.sum	inst	0
smsp__branch_targets_threads_divergent		0

kernelUpperDiagonale

kernelUpperDiagonale(const double *, double *, int, int) (64, 64, 1)x(32, 8, 1), Context 1,
Stream 7, Device 0, CC 8.6

Section: Command line profiler metrics

Metric Name	Metric Unit	Metric Value
gpu__time_duration.sum	usecond	27,78
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.avg		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.max		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.min		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.avg		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.max		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.min		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum		0
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.avg	sector	739,27
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.max	sector	1 183
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.min	sector	347
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum	sector	22 249
l1tex__t_sectors_pipe_lsu_mem_global_op_st.avg	sector	185,71
l1tex__t_sectors_pipe_lsu_mem_global_op_st.max	sector	568
l1tex__t_sectors_pipe_lsu_mem_global_op_st.min	sector	0
l1tex__t_sectors_pipe_lsu_mem_global_op_st.sum	sector	5 485
sm__sass_inst_executed_op_local.avg	inst	0
sm__sass_inst_executed_op_local.max	inst	0
sm__sass_inst_executed_op_local.min	inst	0
sm__sass_inst_executed_op_local.sum	inst	0
smsp__branch_targets_threads_divergent		0

kernelScaleRhs

kernelScaleRhs(const double *, double *, int) (64, 64, 1)x(32, 8, 1), Context 1, Stream 7,
Device 0, CC 8.6

Section: Command line profiler metrics

Metric Name	Metric Unit	Metric Value
gpu__time_duration.sum	usecond	29,83
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.avg		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.max		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.min		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.avg		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.max		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.min		0
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum		0
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.avg	sector	745,13
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.max	sector	1 235
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.min	sector	325
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum	sector	21 986
l1tex__t_sectors_pipe_lsu_mem_global_op_st.avg	sector	185,07
l1tex__t_sectors_pipe_lsu_mem_global_op_st.max	sector	542
l1tex__t_sectors_pipe_lsu_mem_global_op_st.min	sector	0
l1tex__t_sectors_pipe_lsu_mem_global_op_st.sum	sector	5 398
sm__sass_inst_executed_op_local.avg	inst	0
sm__sass_inst_executed_op_local.max	inst	0
sm__sass_inst_executed_op_local.min	inst	0
sm__sass_inst_executed_op_local.sum	inst	0
smsp__branch_targets_threads_divergent		0

Выводы

В результате выполненной работы были закреплены навыки реализации численных методов линейной алгебры с использованием GPU. Реализован алгоритм нахождения обратной матрицы методом Гаусса–Жордана с частичным выбором главного элемента, что позволило на практике отработать принципы программирования на C++ и CUDA, работу с разреженными и плотными данными в линейной памяти, а также использование библиотеки Thrust на стороне устройства.

Реализация, выполненная в рамках данной работы, носит учебный характер и предназначена для демонстрации принципов параллельной обработки матриц на графическом процессоре. Несмотря на то, что код может быть дополнительно оптимизирован (улучшение шаблона обращения к памяти, использование разделяемой

памяти, более точный подбор конфигурации сетки и блоков), текущий вариант наглядно показывает, как классический последовательный метод Гаусса–Жордана преобразуется в набор GPU-ядер и как организовать обмен данными между хостом и устройством.

На тестовых данных GPU-реализация демонстрирует заметное преимущество по скорости по сравнению с последовательным вариантом на CPU, особенно при росте порядка матрицы, за счёт массового параллелизма и одновременной обработки строк/столбцов. CPU-вариант в этом контексте может рассматриваться в основном как эталон корректности и базовая точка сравнения по производительности.

Метод Гаусса–Жордана и его модификации применяются в задачах решения систем линейных уравнений, вычисления обратных матриц в численных методах, обработке сигналов и изображений, а также в некоторых алгоритмах машинного обучения. Разработанная программа даёт практическое представление о том, как подобные алгоритмы переносить на архитектуру GPU и какие аспекты (выбор главного элемента, устойчивость, организация памяти) критичны для корректной и быстрой работы.

Анализ профилировщика показал, что основные ядра алгоритма имеют небольшое время выполнения: $\text{kernelSwapRows} \approx 31$ мкс, $\text{kernelUnderDiagonale} \approx 39$ мкс, $\text{kernelUpperDiagonale} \approx 28$ мкс и $\text{kernelScaleRhs} \approx 30$ мкс. Видно, что каждое ядро выполняет довольно много обращений к памяти (порядка 22 тысяч). Это говорит о том, что текущее выполнение в основном ограничено скоростью доступа к памяти, а не вычислительной нагрузкой. Во всех ядрах отсутствуют конфликты банков разделяемой памяти и обращения к локальной памяти, а также не наблюдается дивергенции ветвлений, что указывает на корректный выбор схемы параллелизма и выровненный шаблон доступа потоков.